

Engenharia de Software – PUC Campinas

# Algoritmos e Linguagens de Programação

*Prof. Dr. Paulo Cesar de Alvarenga Lucci*

## Ponteiros

# Ponteiros

## ○ O QUE SÃO PONTEIROS?

- Ponteiros são variáveis que armazenam **endereços de memória**.
  - Ao invés de guardar um **valor** diretamente, guardam a **localização** onde o valor está armazenado.
  - Permitem acesso direto e manipulação da memória.

## ○ DECLARAÇÃO DE PONTEIROS

- `int *myIntPtr = (int *)malloc(sizeof(int));` // ponteiro para int
- `char *myCharPointer = (char *)malloc(sizeof(char));` // ponteiro para 1 caracter
- O `*` indica que a variável é um ponteiro.
- O tipo do ponteiro deve corresponder ao tipo da variável que ele aponta.
- O **malloc** em C é uma função usada para alocar memória dinâmica durante a execução do programa.
  - Precisa do `#include <stdlib.h>`.
  - Ela reserva um bloco de memória do tamanho especificado e retorna um ponteiro para o início desse bloco.
  - Se não conseguir alocar, retorna **NULL**.

# Ponteiros

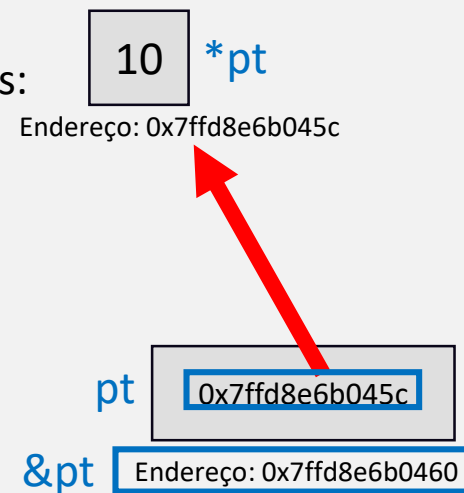
## ○ OPERADORES DE PONTEIROS

- Após declarado, podemos operar sobre os ponteiros através dos seguintes operadores:

- `&`: Endereço (referência) de memória de uma variável ou ponteiro.
- `*`: Conteúdo armazenado no endereço apontado.

- Exemplo:

- `int x = 10; //inteiro x com valor 10`
- `int *pt1 = &x; //ponteiro de inteiro pt1 que aponta para o endereço da variável x. Note que não precisou do malloc.`
- `printf("%d", *pt1); // imprime o valor guardado pelo ponteiro pt1 (que é 10).`
- `int *pt2 = pt1; //ponteiro de inteiro pt3 que aponta para o mesmo endereço do ponteiro pt1, ou seja, a variável x.`
- `printf("%d", *pt2); // imprime o valor guardado pelo ponteiro pt2 (que é 10).`



# Ponteiros

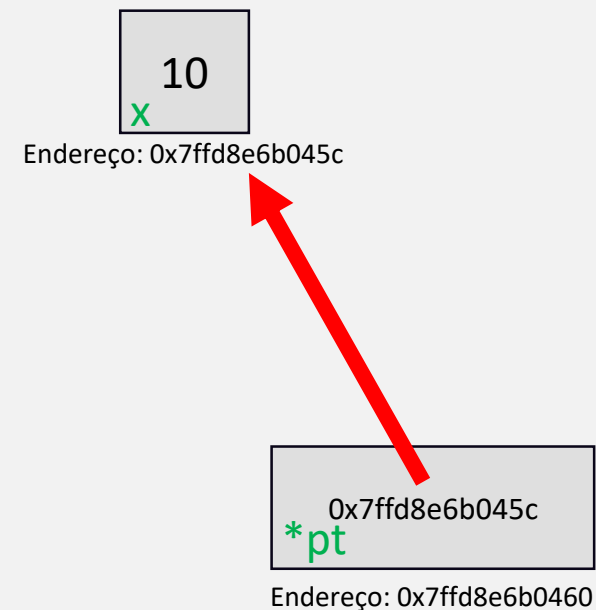
## ○ OPERADORES DE PONTEIROS

```
main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int x = 10;
6      int *pt = &x;
7
8      printf("O valor de x eh %d\n", x);
9      printf("O endereço de x eh %p\n", &x);
10     printf("O endereço do ponteiro pt eh %p\n", &pt);
11     printf("O endereço para onde pt aponta eh %p\n", pt);
12     printf("O valor guardado no endereço para onde pt aponta eh %d\n", *pt);
13
14     return 0;
15 }
```

input

```
O valor de x eh 10
O endereço de x eh 0x7ffd8e6b045c
O endereço do ponteiro pt eh 0x7ffd8e6b0460
O endereço para onde pt aponta eh 0x7ffd8e6b045c
O valor guardado no endereço para onde pt aponta eh 10

...Program finished with exit code 0
Press ENTER to exit console.
```



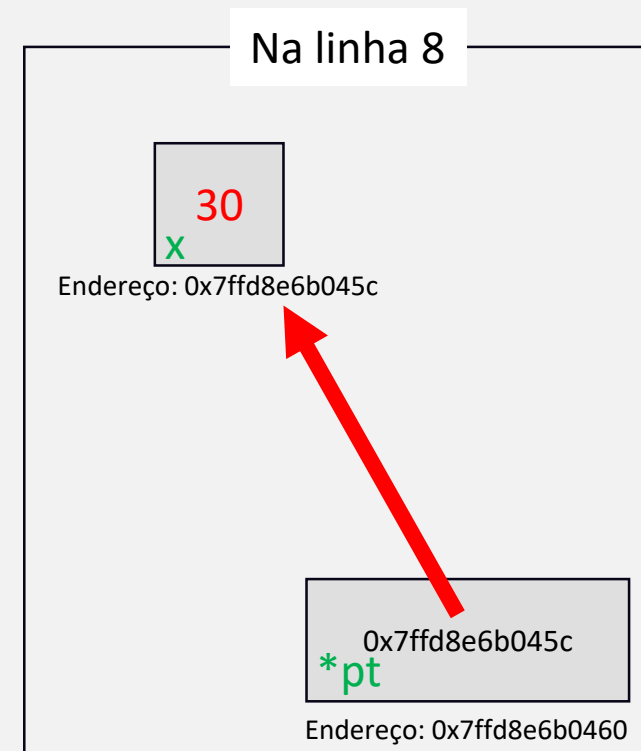
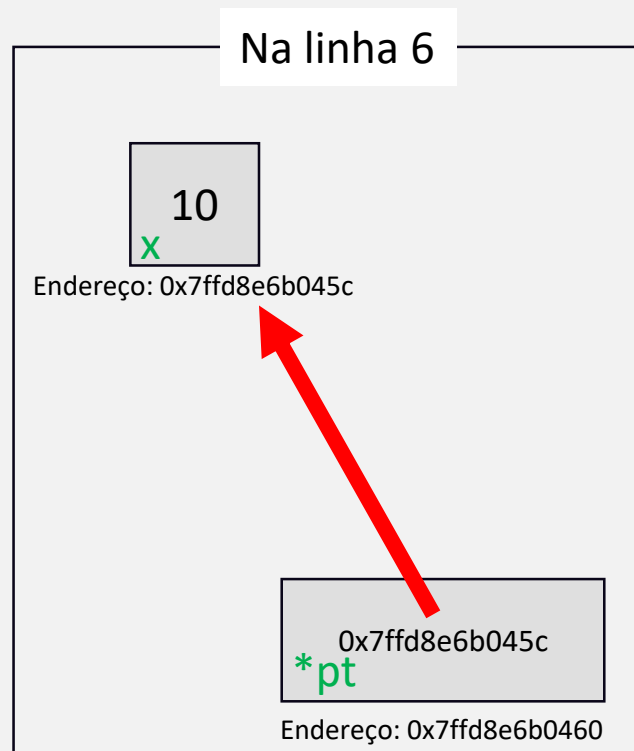
# Ponteiros

## ○ BRINCANDO COM PONTEIROS

```
main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int x = 10;
6      int *pt = &x;
7
8      *pt = 30;
9
10     printf("x era 10 agora eh %d\n", x);
11
12     return 0;
13 }
```

x era 10 agora eh 30

...Program finished with exit code 0  
Press ENTER to exit console.



# Ponteiros

## ○ BRINCANDO COM PONTEIROS

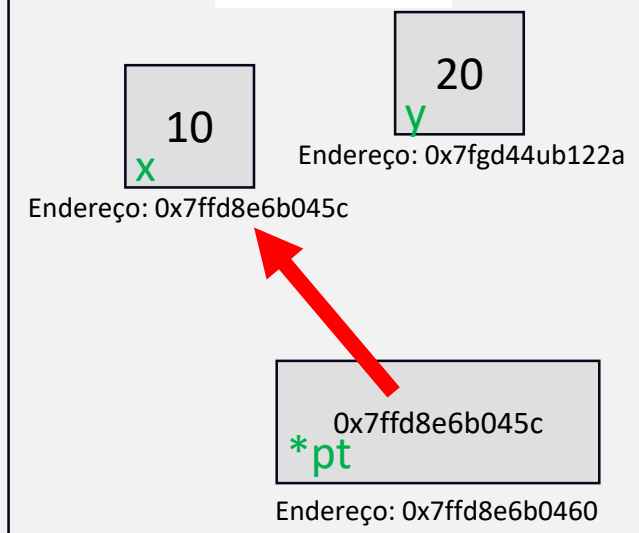
main.c

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int x = 10, y = 20;
6     int *pt = &x;
7
8     *pt = 30;
9     pt = &y;
10    *pt = 2;
11
12    printf("x era 10 agora eh %d\n", x);
13    printf("y era 20 agora eh %d\n", y);
14
15    return 0;
16 }
```

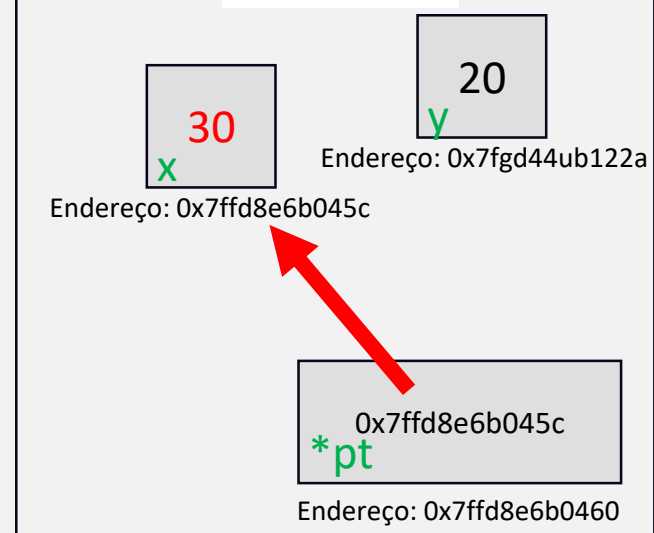
```
x era 10 agora eh 30
y era 20 agora eh 2
```

```
...Program finished with exit code 0
Press ENTER to exit console.□
```

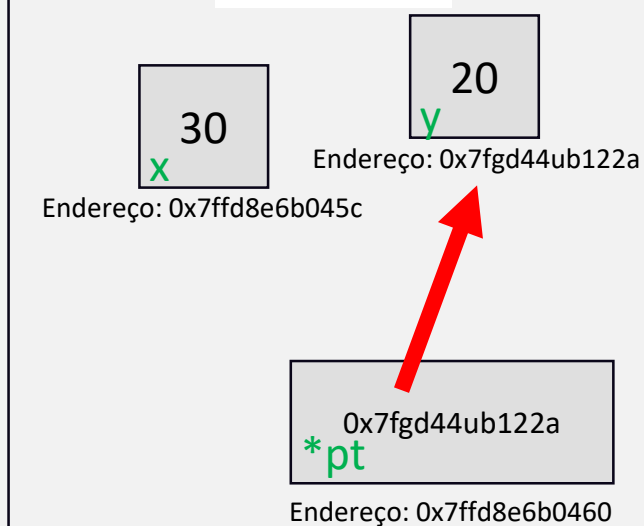
Na linha 6



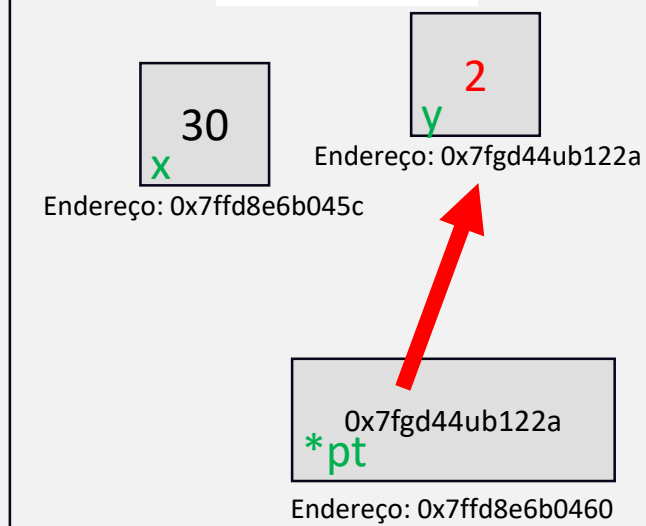
Na linha 8



Na linha 9

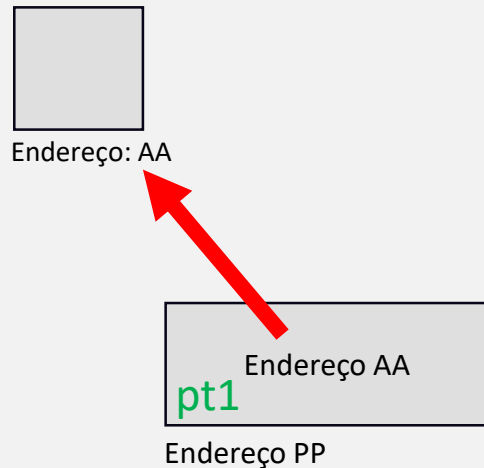


Na linha 10



# Ponteiros

## ○ BRINCANDO COM PONTEIROS



```
main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      int *pt1 = (int *)malloc(sizeof(int));
7      *pt1 = 5;
8
9      int val1 = *pt1;
10     *pt1 = 10;
11
12     printf("Valor guardado por pt1: %d\n", *pt1);
13     printf("Valor guardado por val1: %d\n\n", val1);
14
15     int *pt2 = pt1;
16     *pt2 = 15;
17
18     int val2 = 20;
19     pt2 = &val2;
20     *pt2 = 25;
21
22     printf("Valor guardado por pt1: %d\n", *pt1);
23     printf("Valor guardado por val1: %d\n\n", val1);
24     printf("Valor guardado por pt2: %d\n", *pt2);
25     printf("Valor guardado por val2: %d\n\n", val2);
26
27     return 0;
28 }
```

```
Valor guardado por pt1: 10
Valor guardado por val1: 5

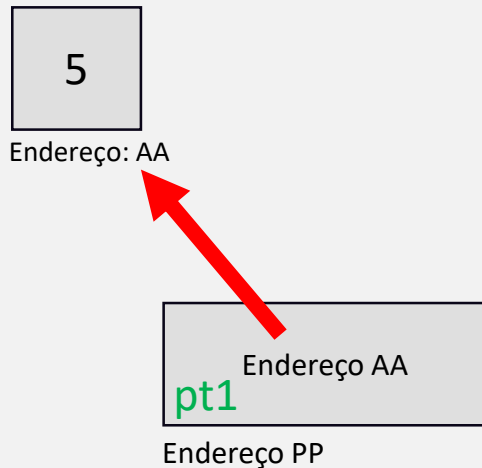
Valor guardado por pt1: 15
Valor guardado por val1: 5

Valor guardado por pt2: 25
Valor guardado por val2: 25

...Program finished with exit code 0
Press ENTER to exit console.
```

# Ponteiros

## ○ BRINCANDO COM PONTEIROS



```
main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      int *pt1 = (int *)malloc(sizeof(int));
7      *pt1 = 5;
8
9      int val1 = *pt1;
10     *pt1 = 10;
11
12     printf("Valor guardado por pt1: %d\n", *pt1);
13     printf("Valor guardado por val1: %d\n\n", val1);
14
15     int *pt2 = pt1;
16     *pt2 = 15;
17
18     int val2 = 20;
19     pt2 = &val2;
20     *pt2 = 25;
21
22     printf("Valor guardado por pt1: %d\n", *pt1);
23     printf("Valor guardado por val1: %d\n\n", val1);
24     printf("Valor guardado por pt2: %d\n", *pt2);
25     printf("Valor guardado por val2: %d\n\n", val2);
26
27     return 0;
28 }
```

```
Valor guardado por pt1: 10
Valor guardado por val1: 5

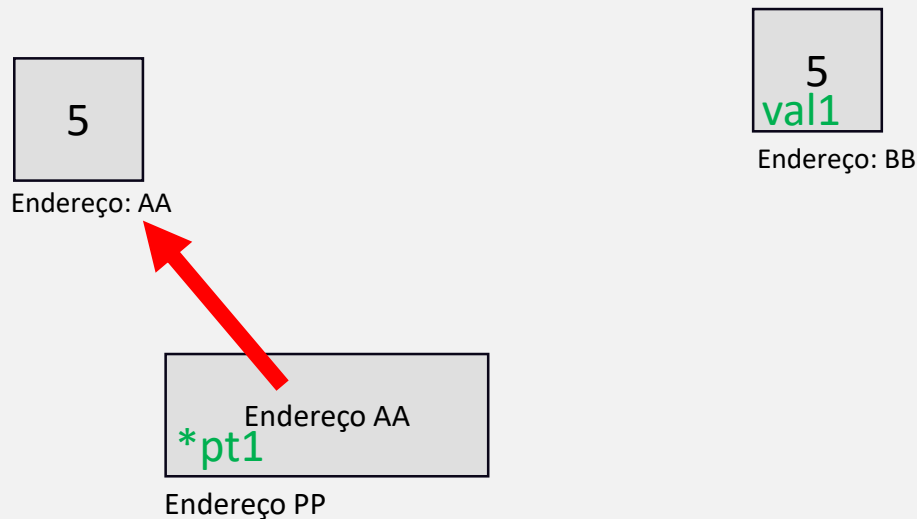
Valor guardado por pt1: 15
Valor guardado por val1: 5

Valor guardado por pt2: 25
Valor guardado por val2: 25

...Program finished with exit code 0
Press ENTER to exit console.
```

# Ponteiros

## ○ BRINCANDO COM PONTEIROS



```
main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      int *pt1 = (int *)malloc(sizeof(int));
7      *pt1 = 5;
8
9      int val1 = *pt1;
10     *pt1 = 10;
11
12     printf("Valor guardado por pt1: %d\n", *pt1);
13     printf("Valor guardado por val1: %d\n\n", val1);
14
15     int *pt2 = pt1;
16     *pt2 = 15;
17
18     int val2 = 20;
19     pt2 = &val2;
20     *pt2 = 25;
21
22     printf("Valor guardado por pt1: %d\n", *pt1);
23     printf("Valor guardado por val1: %d\n\n", val1);
24     printf("Valor guardado por pt2: %d\n", *pt2);
25     printf("Valor guardado por val2: %d\n\n", val2);
26
27     return 0;
28 }
```

```
Valor guardado por pt1: 10
Valor guardado por val1: 5

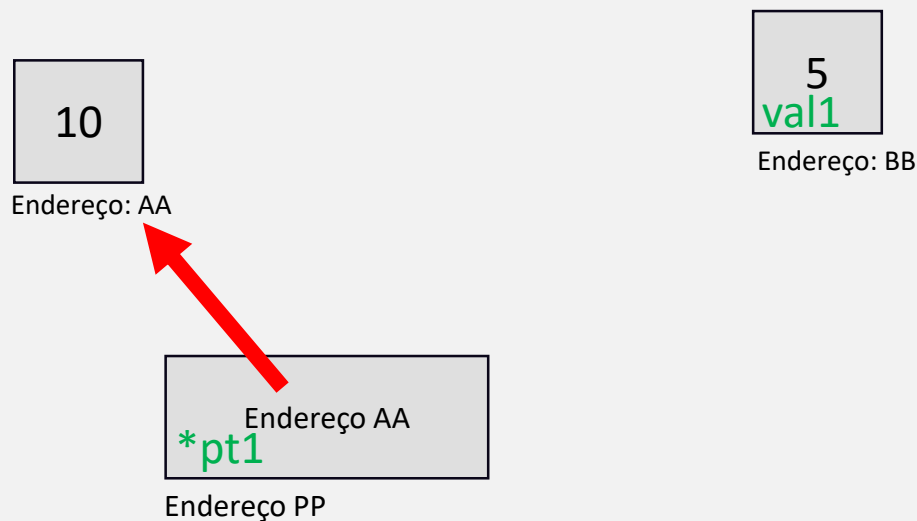
Valor guardado por pt1: 15
Valor guardado por val1: 5

Valor guardado por pt2: 25
Valor guardado por val2: 25
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

# Ponteiros

## ○ BRINCANDO COM PONTEIROS



```
main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      int *pt1 = (int *)malloc(sizeof(int));
7      *pt1 = 5;
8
9      int val1 = *pt1;
10     *pt1 = 10;
11
12     { printf("Valor guardado por pt1: %d\n", *pt1);
13       printf("Valor guardado por val1: %d\n\n", val1);
14     }
15
16     int *pt2 = pt1;
17     *pt2 = 15;
18
19     int val2 = 20;
20     pt2 = &val2;
21     *pt2 = 25;
22
23     printf("Valor guardado por pt1: %d\n", *pt1);
24     printf("Valor guardado por val1: %d\n\n", val1);
25     printf("Valor guardado por pt2: %d\n", *pt2);
26     printf("Valor guardado por val2: %d\n\n", val2);
27
28     return 0;
29 }
```

```
Valor guardado por pt1: 10
Valor guardado por val1: 5
```

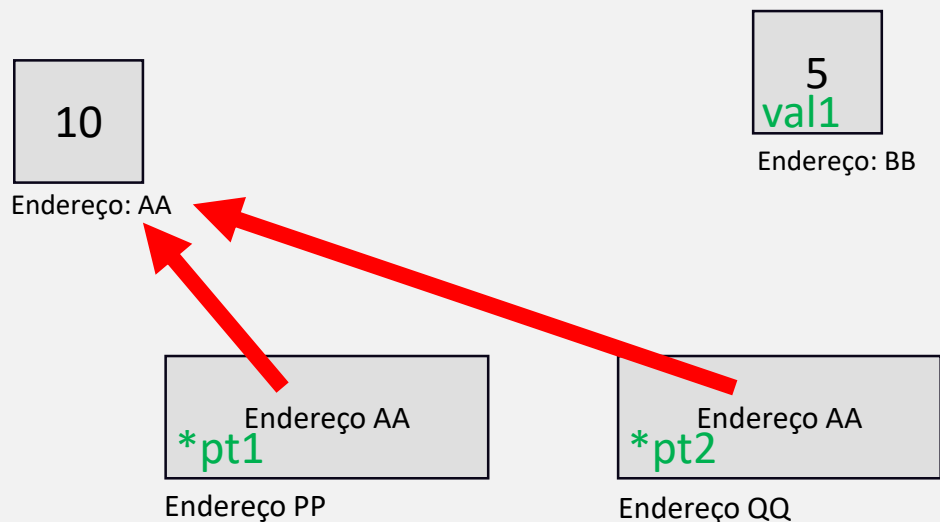
```
Valor guardado por pt1: 15
Valor guardado por val1: 5
```

```
Valor guardado por pt2: 25
Valor guardado por val2: 25
```

```
...Program finished with exit code 0
Press ENTER to exit console.□
```

# Ponteiros

## ○ BRINCANDO COM PONTEIROS



main.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     int *pt1 = (int *)malloc(sizeof(int));
7     *pt1 = 5;
8
9     int val1 = *pt1;
10    *pt1 = 10;
11
12    printf("Valor guardado por pt1: %d\n", *pt1);
13    printf("Valor guardado por val1: %d\n\n", val1);
14
15    int *pt2 = pt1;
16    *pt2 = 15;
17
18    int val2 = 20;
19    pt2 = &val2;
20    *pt2 = 25;
21
22    printf("Valor guardado por pt1: %d\n", *pt1);
23    printf("Valor guardado por val1: %d\n\n", val1);
24    printf("Valor guardado por pt2: %d\n", *pt2);
25    printf("Valor guardado por val2: %d\n\n", val2);
26
27    return 0;
28 }
```

```
Valor guardado por pt1: 10
Valor guardado por val1: 5

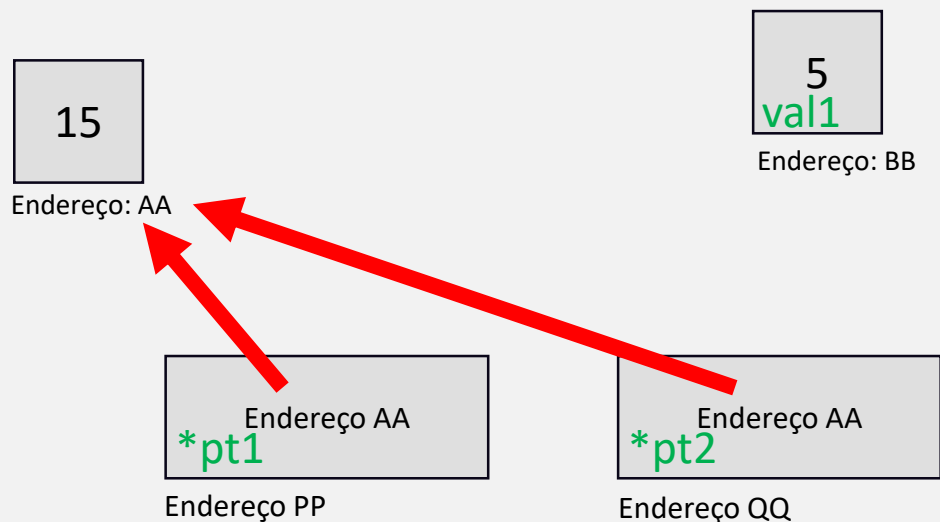
Valor guardado por pt1: 15
Valor guardado por val1: 5

Valor guardado por pt2: 25
Valor guardado por val2: 25
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

# Ponteiros

## ○ BRINCANDO COM PONTEIROS



main.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     int *pt1 = (int *)malloc(sizeof(int));
7     *pt1 = 5;
8
9     int val1 = *pt1;
10    *pt1 = 10;
11
12    printf("Valor guardado por pt1: %d\n", *pt1);
13    printf("Valor guardado por val1: %d\n\n", val1);
14
15    int *pt2 = pt1;
16    *pt2 = 15;
17
18    int val2 = 20;
19    pt2 = &val2;
20    *pt2 = 25;
21
22    printf("Valor guardado por pt1: %d\n", *pt1);
23    printf("Valor guardado por val1: %d\n\n", val1);
24    printf("Valor guardado por pt2: %d\n", *pt2);
25    printf("Valor guardado por val2: %d\n\n", val2);
26
27    return 0;
28 }
```

```
Valor guardado por pt1: 10
Valor guardado por val1: 5

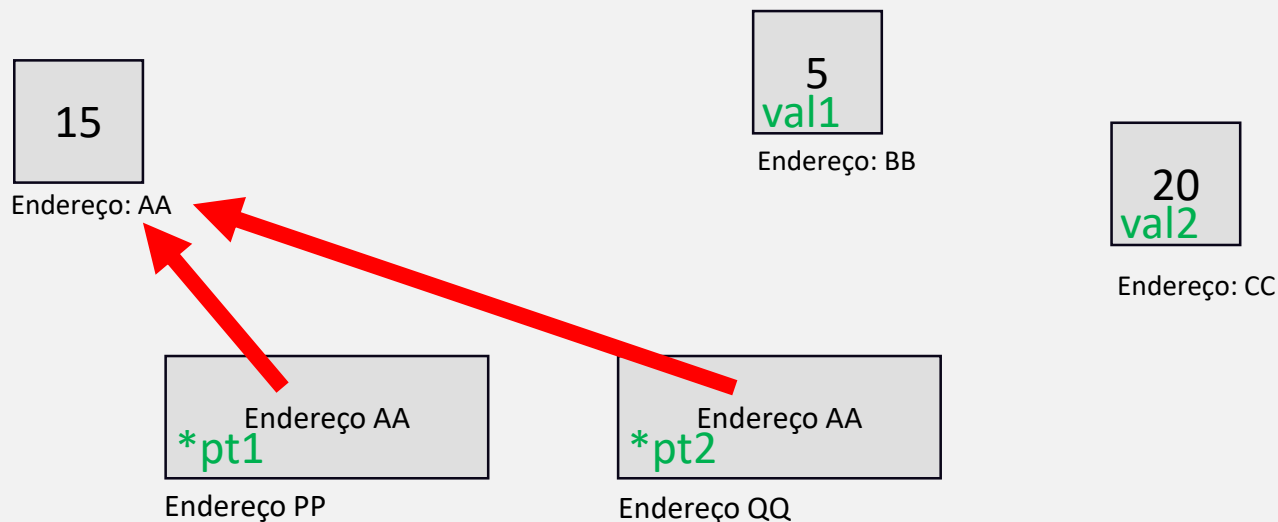
Valor guardado por pt1: 15
Valor guardado por val1: 5

Valor guardado por pt2: 25
Valor guardado por val2: 25
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

# Ponteiros

## ○ BRINCANDO COM PONTEIROS



main.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     int *pt1 = (int *)malloc(sizeof(int));
7     *pt1 = 5;
8
9     int val1 = *pt1;
10    *pt1 = 10;
11
12    printf("Valor guardado por pt1: %d\n", *pt1);
13    printf("Valor guardado por val1: %d\n\n", val1);
14
15    int *pt2 = pt1;
16    *pt2 = 15;
17
18    int val2 = 20;
19    pt2 = &val2;
20    *pt2 = 25;
21
22    printf("Valor guardado por pt1: %d\n", *pt1);
23    printf("Valor guardado por val1: %d\n\n", val1);
24    printf("Valor guardado por pt2: %d\n", *pt2);
25    printf("Valor guardado por val2: %d\n\n", val2);
26
27    return 0;
28 }
```

```
Valor guardado por pt1: 10
Valor guardado por val1: 5

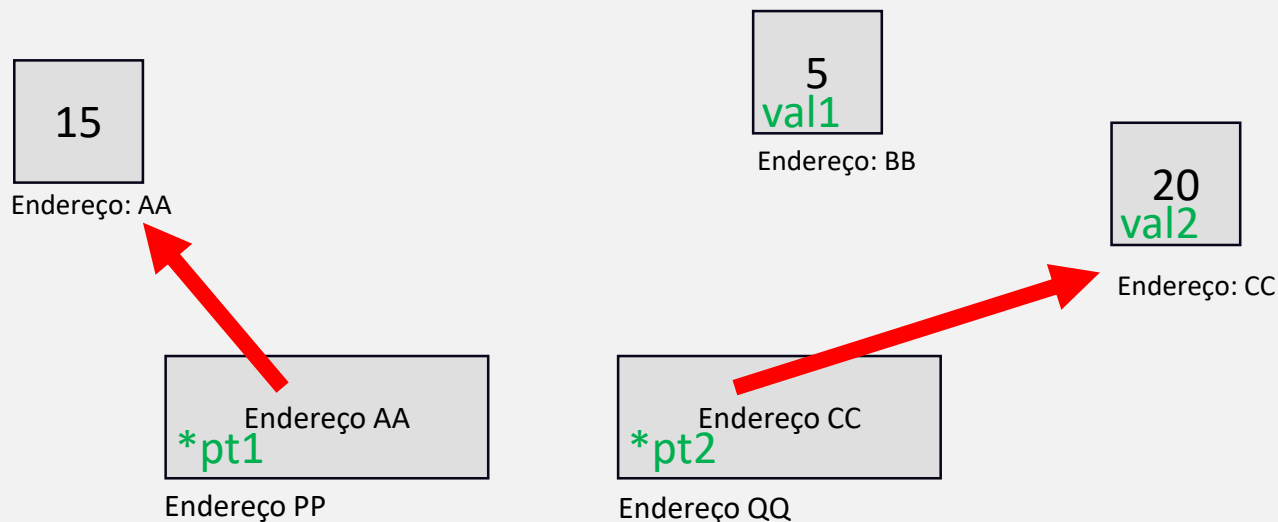
Valor guardado por pt1: 15
Valor guardado por val1: 5

Valor guardado por pt2: 25
Valor guardado por val2: 25
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

# Ponteiros

## ○ BRINCANDO COM PONTEIROS



main.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     int *pt1 = (int *)malloc(sizeof(int));
7     *pt1 = 5;
8
9     int val1 = *pt1;
10    *pt1 = 10;
11
12    printf("Valor guardado por pt1: %d\n", *pt1);
13    printf("Valor guardado por val1: %d\n\n", val1);
14
15    int *pt2 = pt1;
16    *pt2 = 15;
17
18    int val2 = 20;
19    pt2 = &val2;
20    *pt2 = 25;
21
22    printf("Valor guardado por pt1: %d\n", *pt1);
23    printf("Valor guardado por val1: %d\n\n", val1);
24    printf("Valor guardado por pt2: %d\n", *pt2);
25    printf("Valor guardado por val2: %d\n\n", val2);
26
27    return 0;
28 }
```

```
Valor guardado por pt1: 10
Valor guardado por val1: 5

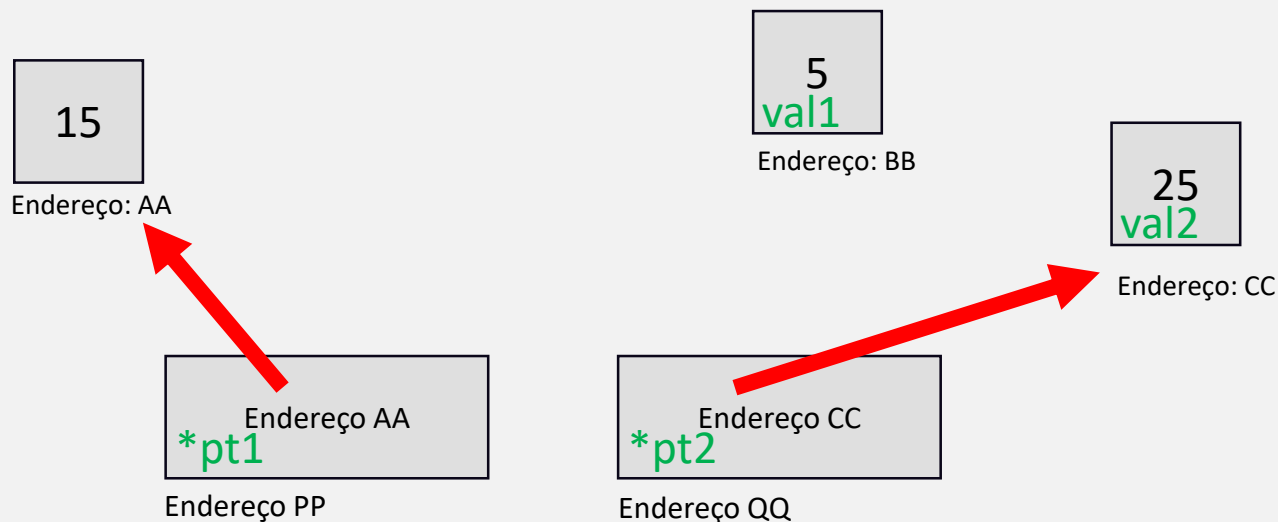
Valor guardado por pt1: 15
Valor guardado por val1: 5

Valor guardado por pt2: 25
Valor guardado por val2: 25
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

# Ponteiros

## ○ BRINCANDO COM PONTEIROS



```
main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      int *pt1 = (int *)malloc(sizeof(int));
7      *pt1 = 5;
8
9      int val1 = *pt1;
10     *pt1 = 10;
11
12     printf("Valor guardado por pt1: %d\n", *pt1);
13     printf("Valor guardado por val1: %d\n\n", val1);
14
15     int *pt2 = pt1;
16     *pt2 = 15;
17
18     int val2 = 20;
19     pt2 = &val2;
20     *pt2 = 25;
21
22     { printf("Valor guardado por pt1: %d\n", *pt1);
23       printf("Valor guardado por val1: %d\n\n", val1);
24       printf("Valor guardado por pt2: %d\n", *pt2);
25       printf("Valor guardado por val2: %d\n\n", val2);
26     }
27     return 0;
28 }
```

```
Valor guardado por pt1: 10
Valor guardado por val1: 5

Valor guardado por pt1: 15
Valor guardado por val1: 5

Valor guardado por pt2: 25
Valor guardado por val2: 25
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

# Ponteiros

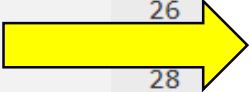
## ○ CUIDADOS COM PONTEIROS

- Toda memória **alocada dinamicamente** permanece reservada até ser liberada.
- O sistema não libera automaticamente após o uso.
  - O ponteiro **alocado dinamicamente** que não será mais utilizado, **se não for liberado da memória**, ficará com sua alocação reservada a ele até que o programa **termine por completo**, consumindo memória RAM (*memory leak*).
  - Nos exemplos anteriores não foi feita a liberação explícita dos ponteiros alocados dinamicamente pois estavam perto do término do programa (nas últimas linhas do main).
    - Em um programa de grandes proporções, essa liberação se torna obrigatória.
- Para evitar vazamento de memória (*memory leak*), use:
  - **free(ptr);**
  - **ptr = NULL;**

# Ponteiros

## ○ REFAZENDO O EXEMPLO ANTERIOR

```
main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      int *pt1 = (int *)malloc(sizeof(int));
7      *pt1 = 5;
8
9      int val1 = *pt1;
10     *pt1 = 10;
11
12     printf("Valor guardado por pt1: %d\n", *pt1);
13     printf("Valor guardado por val1: %d\n\n", val1);
14
15     int *pt2 = pt1;
16     *pt2 = 15;
17
18     int val2 = 20;
19     pt2 = &val2;
20     *pt2 = 25;
21
22     printf("Valor guardado por pt1: %d\n", *pt1);
23     printf("Valor guardado por val1: %d\n\n", val1);
24     printf("Valor guardado por pt2: %d\n", *pt2);
25     printf("Valor guardado por val2: %d\n\n", val2);
26
27     free(pt1);
28     pt1 = NULL;
29
30     //somente p1 está apontando para memória heap (alocada pelo malloc).
31     //Por essa razão fazemos free(pt1).
32     //pt2 não aponta pra memória alocada, mas sim para memória stack.
33     //Então não precisa fazer free(pt2).
34
35     return 0;
36 }
```



# Ponteiros

## ○ EXEMPLO

```
main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      // Aloca 1 inteiro na heap
6      int *pt = (int*)malloc(sizeof(int));
7      *pt = 42;
8      printf("Valor alocado na heap: %d\n", *pt);
9
10     // Libera a memória da heap antes de sobrescrever o ponteiro
11     free(pt);
12     pt = NULL; // boa prática: evitar ponteiro solto
13
14     // Agora podemos reutilizar o ponteiro com segurança
15     int val = 99;
16     pt = &val;
17     printf("Agora pt aponta para a stack: %d\n", *pt);
18
19     // Não dar free aqui, pois pt não aponta mais para malloc
20     return 0;
21 }
```

▼ ↗ 📄 ⚙️ 🗑️

```
Valor alocado na heap: 42
Agora pt aponta para a stack: 99

...Program finished with exit code 0
Press ENTER to exit console.
```